Arrays

FUNDAMENTALS OF PROGRAMMING

Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

Plan for today

- Arrrays and one-dimensional arrays
- Two-dimensional arrays

Arrays &
One-dimensional Arrays

Motivation

- A program that requires to store 3 integers?

=> declare and use 3 integer variables a1, a2, a3;

- A program that requires to store 100 integers?

=> declare and use 100 integer variables (feasibility?)

Users want to input n integers?

Solution

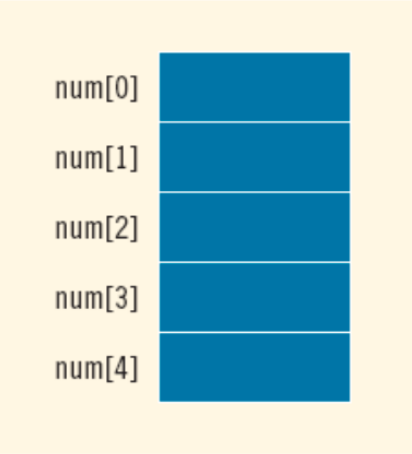
A new data type that allows to store a series of integers and fast access

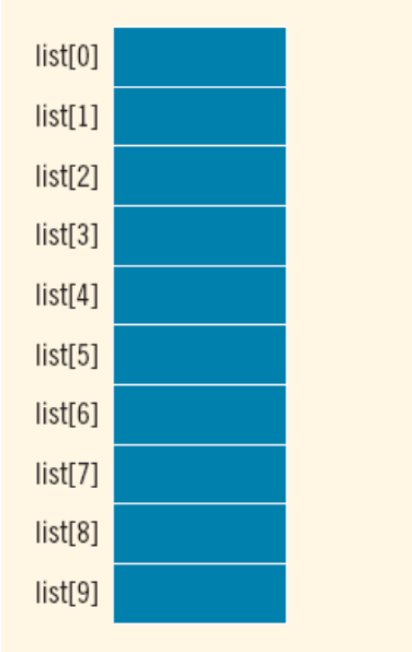
Arrays

- A collection of items stored at **contiguous memory locations**
- All elements must be the **same types**. For example: an integer array, an array of characters
- Elements can be **accessed randomly** using **indices** of an array
- Used to represent **many instances** in **one variable**

Example

```
int num[5];
int list[10];
```





Declaration

```
<DataType> Name[<SIZE>];
<DataType> Name[<SIZE1>][<SIZE2>]...[<SIZEN>];
```

`<SIZE1>, <SIZE2>, … <SIZEN>` are sizes of 1st, 2nd, …, Nth dimensional-elements

Note:

- `<DataType>:` `int, float, double, …`
- Determine **size** (should use **const**) of arrays when declaring
- Multi dimensional arrays: nEles N = `SIZE1*SIZE2*…*SIZEN`
- Memory size = N * `sizeof(<DataType>)`
- One-dimensional arrays, array indices start from 0 to `<SIZE>`**-1**

Examples

```
int iArray1[100]; // num of elements & memory size?

const int n = 20;
int iArray2[n];   // num of elements & memory size?

const int m = 10;
int iArray3[m];   // num of elements & memory size?
```

Array initialization

- Initializing values of elements for **all** elements in an array

```
int arr[4] = {2912, 1706, 1506, 1904};
```

- Initializing values of elements for **some first** elements in an array

```
int arr[4] = {2912, 1706};
```

Array initialization

- Initializing zero values for all elements in an array

```
int arr[4] = {0};
```

- Compiler can determine the size of an array via initialized values

```
int arr[] = {2912, 1706, 1506, 1904};
```

Access Elements in Arrays

- Via **indices** (numbers)

`Name[<Index1>][<Index2>]...[<IndexN>]`

e.g., `a[1][5], a[0], a[n-1][5] , a[1][5][3]`

```
int a[4];
```

Access elements in arrays

valid: `a[0], a[1], a[2], a[3]`

invalid: `a[-1], a[4], a[5],` …

Notes

- **DO NOT** use array = array

```
int ArrayA[4] = {1, 2, 3, 4}, ArrayB[4];
ArrayB = ArrayA;
```

Notes

- **DO NOT** use array = array

```
int ArrayA[4] = {1, 2, 3, 4}, ArrayB[4];
ArrayB = ArrayA; // wrong
```

- Solution:

```
for (int i = 0; i < 4; i++)
    ArrayB[i] = ArrayA[i];
```

Notes

- **DO NOT** use `cin` to get inputs of arrays

```
int Array[4];
cin >> Array;
```

Notes

- **DO NOT** use `cin` to get inputs of arrays

```
int Array[4];
cin >> Array; // wrong
```

- Solution:

```
for (int i = 0; i < 4; i++)
    cin >> Array[i];
```

Notes

- **DO NOT** use `cout` to print out outputs of arrays

```
int Array[4];
cout << Array;
```

Notes

- **DO NOT** use `cout` to print out outputs of arrays

```
int Array[4] = {1, 2, 3, 4};
cout << Array; // Results?
```

- Solution

```
for (int i = 0; i < 4; i++)
    cout << Array[i] << " ";
```

1D Arrays as Function Parameters

- Arrays are passed by **addresses** only
  - We can change values of arrays within a function
- **Do not use the symbol &** when passing arrays to a function
- The size of the array is **usually omitted**
  - If provided, it is ignored by the compiler

1D Arrays as Function Parameters

```
void zeroFill(int arr[], int n) {
    for (int i = 0; i < n; i++)
        arr[i] = 0;
}
void zeroFill(int arr[100], int size) {
    // …
}
void zeroFill(int *arr, int size) {
    // …
}
```

1D Arrays as Function Parameters

- Using `const` to tell that the array elements are not changed in functions.

```
void printArray(const int a[], int n){
    for (int i = 0; i < n; i++)
        cout << "a[" << i << "]: " << a[i] << endl;
}
```

Write functions for 1D arrays

- Get integer inputs and put them in an array a, having a size n
- Print out an array a with a size n
- Find whether an element k in an array a with a size n, if not, return -1
- Check all elements an array a with a size n , are prime numbers or not.
- Move prime numbers from an array a to an array b
- Merge two arrays into one array
- Find a maximum/minimum value in a non-empty array

Write functions for 1D arrays

- Sort an array in ascending/descending order
- Add/remove an element

Two-dimensional arrays

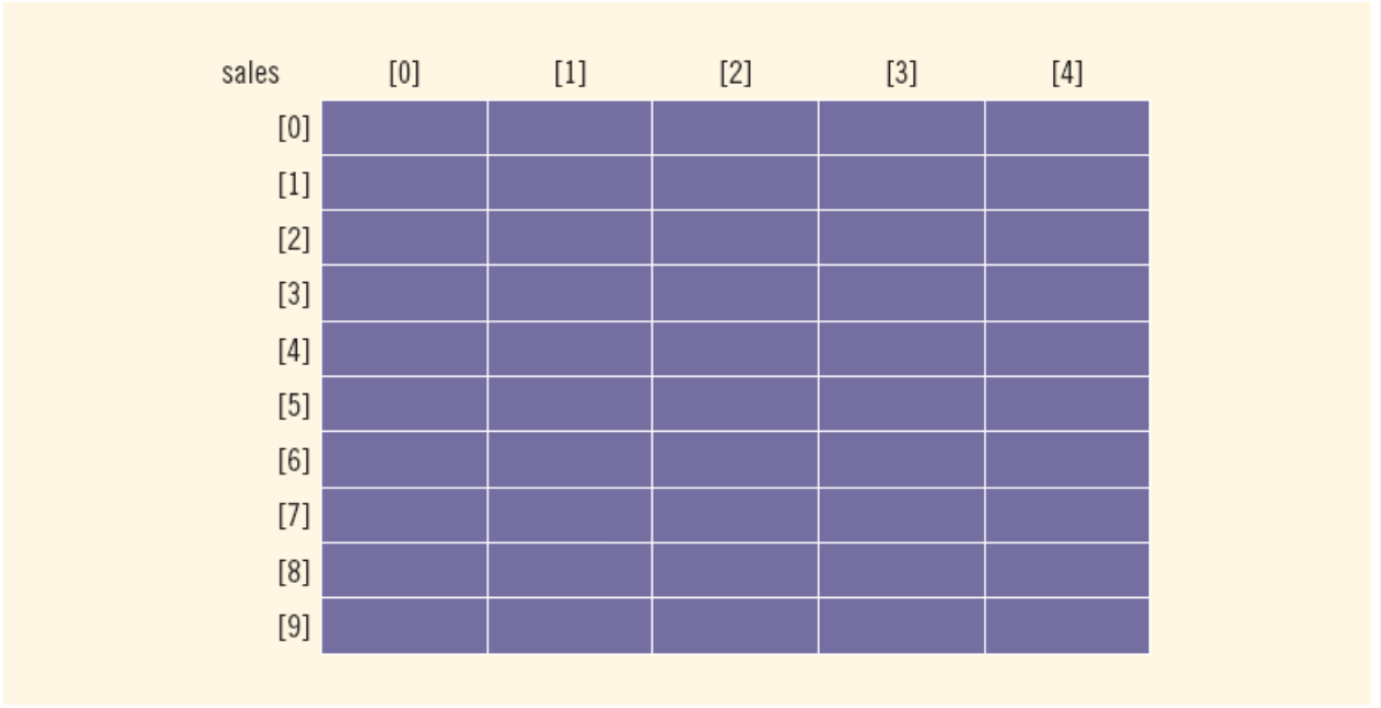
2D Arrays

- A collection of a fixed number of components (of the **same type**) arranged in **two** dimensions
  - Sometimes called matrices or tables

- Values can be **accessed** via indices from **row index** and **column index** of 2D arrays

Example

```
double sales[10][5];
```



Declaration

`<DataType> Name[<SIZE1>][<SIZE2>]`

`<SIZE1>, <SIZE2>` are sizes of 1st and 2nd dimensional-elements

Notes:

`<DataType>:` `int, float, double, …`

Determine `<SIZE1>, <SIZE1>` when declaring

nElements N = ?, memory size = ?

Row index and column index start from 0 to `size`**-1**. E.g., `int arr2D[2][2];`

```
// includes elements: arr2D[0][0] … arr2D[1][1]
```

Examples

```
int Arr2D1[10][10];   // number of elements and
memory size?


const int n = 20;
int Arr2D2[n][n];     // number of elements and
memory size?


const int m = 10;
int Arr2D3[m][n];     // number of elements and
memory size?
```

Array initialization

- Initializing values of elements for **all** elements in a 2D array?

```
int arr2D[2][2] = {{1,2}, {3,4}};
int arr2D[2][2] = {1,2,3,4};
```

- Initializing values of elements for **some first** elements in a 2D array?
- Initializing zero values for all elements in a 2D array?
- Compiler can determine the size of an array via initialized values?

```
int arr2D[][2] = {1,2,3,4}; // number of elements
and memory size?
```

Access Elements in Arrays

- Using row index and column index

```
Name[<RowIndex>][<ColumnIndex>]
```

E.g., `a[1][5], a[0][nCol-1], a[nRow-1][5]`

```
int a[4][3];
```

Access elements in arrays

Valid: `a[0][0], a[0][1], ..., a[3][2]`

Invalid: `a[-1][2], a[4][3], a[3][3], …`

Notes

- **DO NOT** use array = array

```
int Array2DA[2][2] = {{1, 2}, {3, 4}},
    Array2DB[2][2];


Array2DB = Array2DA;
```

Notes

- **DO NOT** use array = array

```
int Array2DA[2][2] = {{1, 2}, {3, 4}},
    Array2DB[2][2];


Array2DB = Array2DA; // wrong
```

- Solution:

```
for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++)
        Array2DB[i][j] = Array2DA[i][j];
```

Notes

- **DO NOT** use `cin` to get inputs of 2D arrays

```
int Array2D[4][4];

cin >> Array2D;
```

Notes

- **DO NOT** use `cin` to get inputs of 2D arrays

```
int Array2D[4][4];


cin >> Array2D; // wrong
```

- Solution:

```
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4; j++)
        cin >> Array2D[i][j];
```

Notes

- **DO NOT** use `cout` to print out outputs of 2D arrays

```
int Array2D[4][4];
cout << Array2D; // what results?
```

- Solution `// print out shape of a matrix`

```
for (int i = 0; i < 4; i++){
    for (int j = 0; j < 4; j++)
        cout << Array2D[i][j] << " ";
    cout << endl;
}
```

2D Arrays as Function Parameters

- 2D Arrays are passed by **addresses** (or **references**) only
  - We can change values of 2D arrays within a function

- **Do not use the symbol &** when passing 2D arrays to a function

- Colum size of 2D arrays must be **determined**
- Row size of 2D arrays is **usually omitted**
  - If provided, it is ignored by the compiler

2D Arrays as Function Parameters

```
void zeroFill(int a[][10], int nRow, int nCol) {
     …
}
void zeroFill(int a[100][100], int m, int n) {
     // …
}
void zeroFill(int *a[100] , int nRow, int nCol){
     // …
}
```

2D Arrays as Function Parameters

- Using `const` to tell that the array elements are not changed in functions.

```
void print2DArray(const int a[][100], int m, int n){
    for (int i = 0; i < m; i++){
        for (int j = 0; j < n; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
}
```

Write functions for 2D arrays

- Get integer inputs and put them in a 2D array a, with m rows & n cols
- Print out a 2D array a with m rows and n columns
- Find whether an element k in an array a with m rows and n columns, if not, return -1
- Print out elements in the main diagonal (\) and opposite diagonal (/) in a matrix n*n
- Print out the transpose matrix, e.g., 2*3 → 3*2
- Print out elements with the shapes of the matrix n*n: upper-half left. upper-half right, lower-half left, lower-half right triangle.
- Print out the sum of two matrices a, b

Other topics

- `typedef` and `#define`

```
typedef int Matrix100x100[100][100];
Matrix100x100 a, b;

#define ROWSIZE 100    // try to avoid
#define COLSIZE 100    // try to avoid
```

Other topics

```cpp
#include <iostream>
using namespace std;

typedef char* ptr;
#define PTR char*
int main(){
    ptr a, b, c;
    PTR x, y, z;
    cout << "size of a: " << sizeof(a) << endl;
    cout << "size of b: " << sizeof(b) << endl;
    cout << "size of c: " << sizeof(c) << endl;
    cout << "size of x: " << sizeof(x) << endl;
    cout << "size of y: " << sizeof(y) << endl;
    cout << "size of z: " << sizeof(z) << endl;
    return 0;
}
```